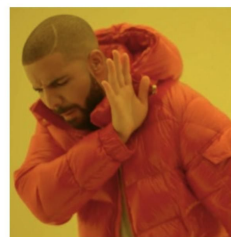


UML Diagrams

Behavioral Diagrams

Vladimir Soroka



Thoughtfully
thinking out
flow of program
and creating UML
diagrams beforehand



"Code first,
ask questions
later" and
restructuring entire
program to
allow for one change

Class Diagram

- Describe the connections between classes in our system

<<Interface>>

Abstract

Class

Attributes

Behaviors

Aggregation



Composition

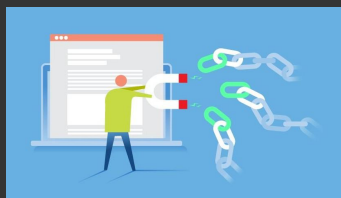


This is a note

Inheritance



+	Public
-	Private
#	Protected



Class Diagram

- Catch the bunny
 - 9x9 board
 - 2 players (a kid and a bunny)
 - Initially, each player is assigned a random square
 - Maximum number of allowed steps is initialized (we call it X)
 - In each turn, a player is allowed to move one square
 - The game ends if
 - The kid catches the bunny
 - I.e., both are placed on the same square
 - The kid wins
 - The kid performs X steps
 - The bunny wins

The computer can move in two different options:

- Random
- Educated

Identify the abstraction

- Game Algorithm
- Board Game
- Catch the bunny game
- What else?

Identify the abstraction

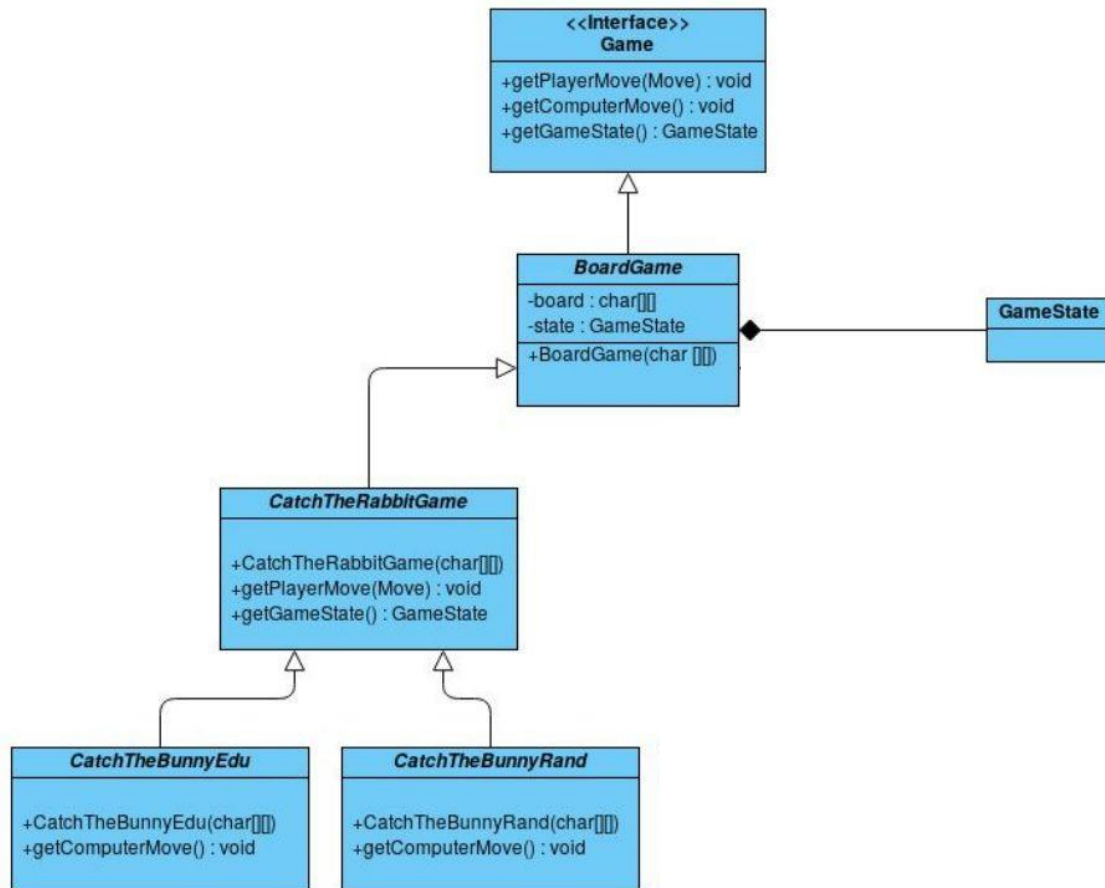
- Game Algorithm
- Board Game
- Catch the bunny game
- What else?
 - Catch the Bunny Educated
 - Catch the Bunny Random

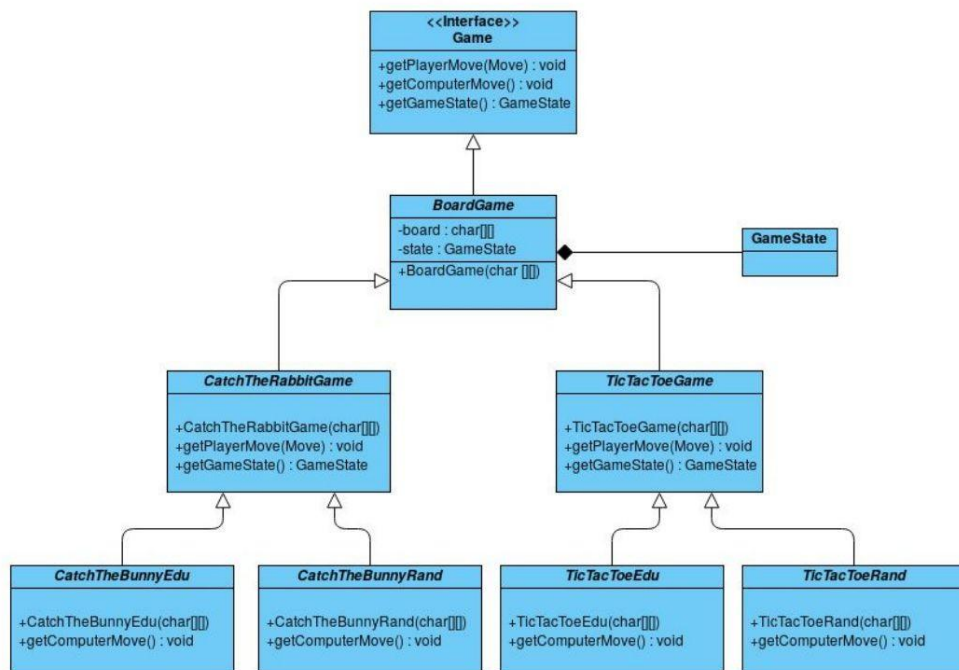
Identify the Relations

- Game Algorithm
 - Compute the next computer move
 - Given coordinate (move) move the player to the given point
 - i. The performed move must be validated
 - Get current game state
 - i. Player wins, computer wins, tie, or in progress
- Board Game (abstract)
 - Generalize all the games that have a board(different size)
 - Implements the game algorithm

Identify the Relations

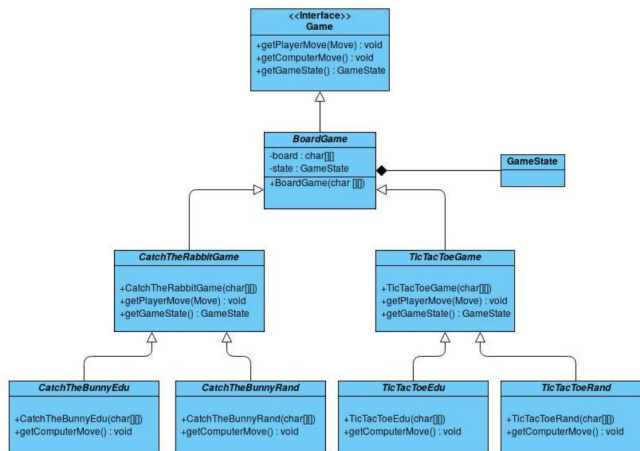
- Catch the bunny game
 - Generalizes the board game
 - Combine the common between all the game variations
- Catch the bunny Random
 - A catch the bunny game in which the computer move is random
- Catch the bunny Educated
 - A catch the bunny game in which the computer move is educated

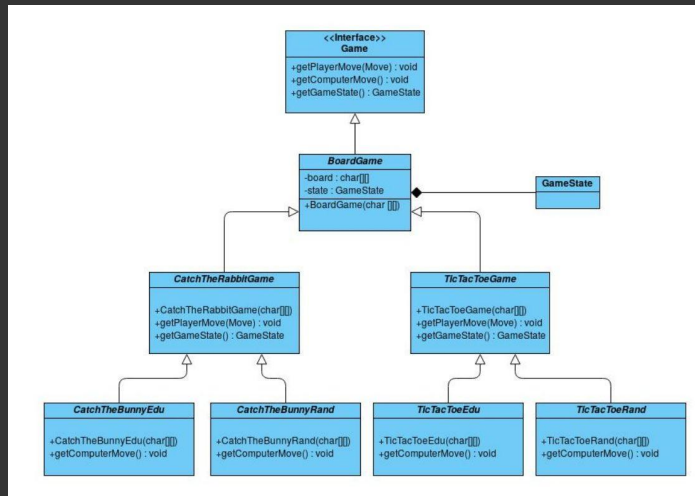
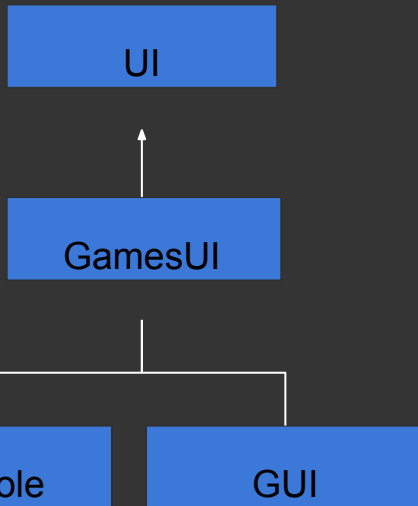


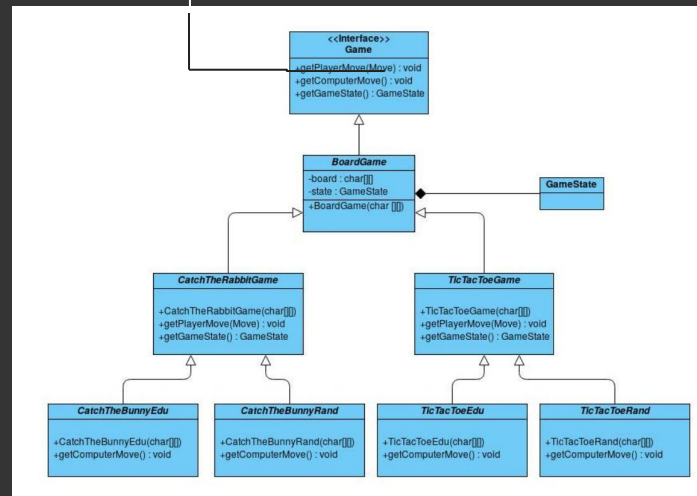
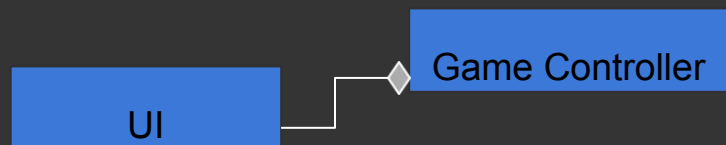


Console

GUI







Behavioral Diagrams

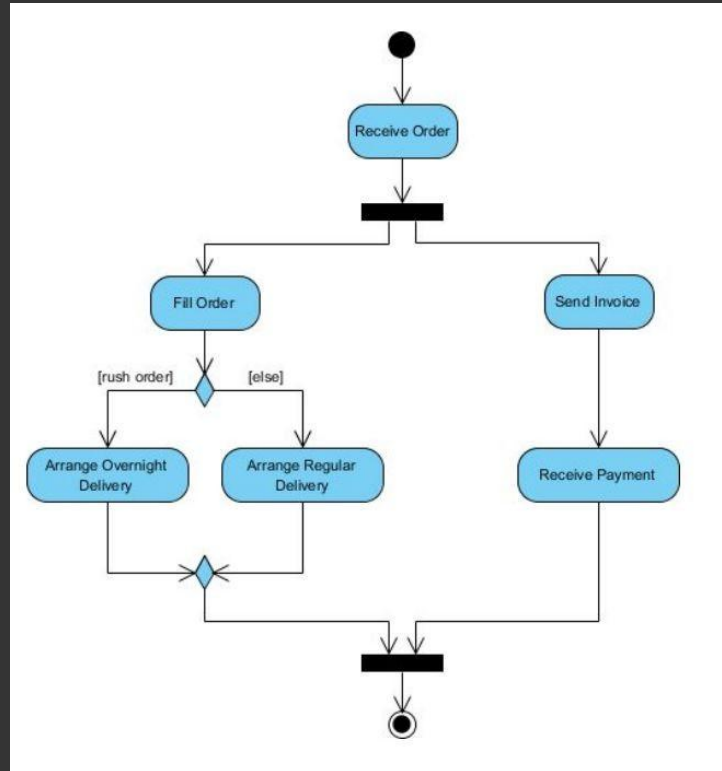
- Behavioral diagrams model how a system behaves over time, focusing on dynamic aspects such as workflows, interactions, and state changes.

◆ Main Types of Behavioral Diagrams		
Diagram Type	Focus	Example Use Case
Use Case Diagram	User goals & system interactions	Booking flights in a travel app
Sequence Diagram	Time-ordered message flow	User resets password via email
Activity Diagram	Workflow and decision points	Online shopping cart and checkout process
State Diagram	State transitions of a system/object	Order lifecycle (new → paid → shipped → done)
Communication Diagram	Object interactions & structure	Chat between multiple users in an app
Interaction Overview Diagram	High-level flow of multiple interactions	Composite user journey in an app
Timing Diagram	Behavior changes over time	Sensor updates in real-time monitoring system

Activity diagram

- Write an activity diagram that:
 - Initial state - Receive order
 - Perform order filling and invoice sending in parallel
 - i. Order fill
 - 1. if it is a rush order arrange an overnight delivery
 - 2. Else arrange regular delivery
 - ii. Send invoice
 - 1. Send the invoice and receive the payment
 - 2. Done

Activity Diagram



Activity diagram (2)

This activity diagram models the behavior of a **user interacting with an online store** in order to buy products. The process involves:

1. **Browsing and selecting products**
2. **Adding them to the cart**
3. **Reviewing the cart**
4. **Proceeding to checkout**
5. **Choosing a payment method**
6. **Placing the order**
7. **Optionally tracking the delivery**

The system may also need to validate user credentials, check product availability, and confirm payment.

Activity Diagram (2)

```

@startuml
|User|
start

:Browse products;
:Select product;

repeat
    :Add product to cart;
    :Continue shopping?;
repeat while (Yes)

:View cart;
:Proceed to checkout;

if (User is logged in?)
then (Yes)
else
    :Log in or register;
endif

|System|
:Validate cart contents;
:Check product
availability;

|User|
:Enter shipping details;
:Choose payment method;

|System|
:Process payment;

if (Payment successful?)
then (Yes)
    :Show confirmation;
    :Send receipt;
    fork
        :Optionally track
        delivery;
        fork again
            :Logout or
            continue shopping;
        end fork
    else
        :Show payment error;
    stop
endif

stop
@enduml

```

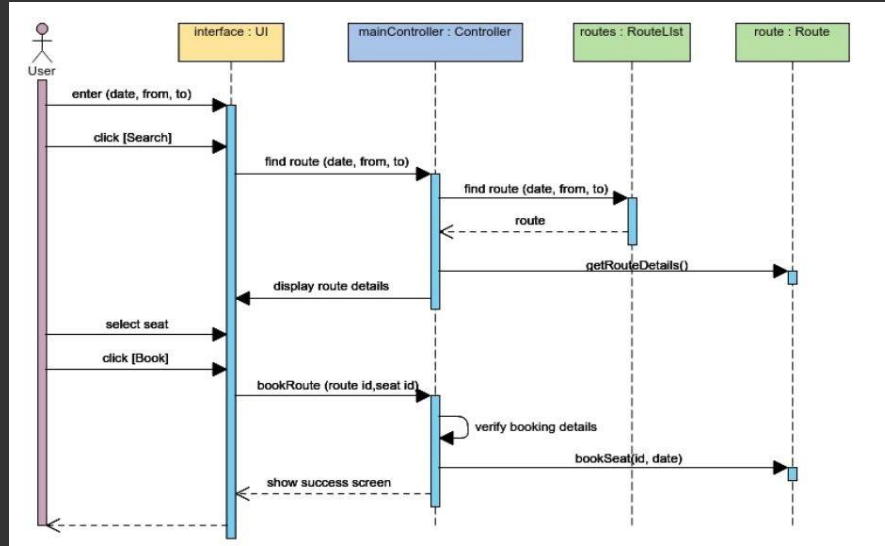


Brought to you by
[PlantUML](https://plantuml.com/)

Sequence Diagram

- Describe a Sequence of a user that would like to book a flight...

Sequence Diagram



Sequence Diagram (2)

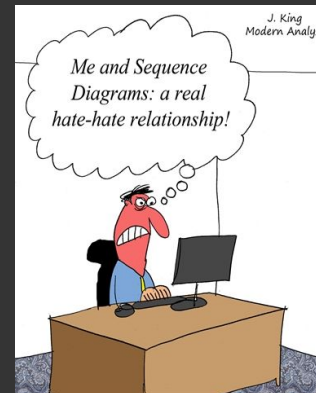
- A user forgot their password and initiates a password reset through the website. The system sends a reset email, the user clicks the link, and the password gets updated.

Sequence Diagram (2)

```
@startuml
actor User
participant "Web Browser" as Browser
participant "Web Server" as Server
participant "Email Service" as Email
database "User Database" as DB

User -> Browser: Go to 'Forgot Password'
Browser -> Server: POST /reset-request (email)
Server -> DB: Check if email exists
DB --> Server: Email found
Server -> Server: Generate token
Server -> DB: Store token
Server -> Email: Send reset link
Email --> User: Email with link

User -> Browser: Click reset link
Browser -> Server: GET /reset-password?token=abc123
Server -> DB: Validate token
DB --> Server: Token valid
Server -> Browser: Show reset form
User -> Browser: Submit new password
Browser -> Server: POST /reset-password
Server -> DB: Update password
Server -> Browser: Show success message
@enduml
```

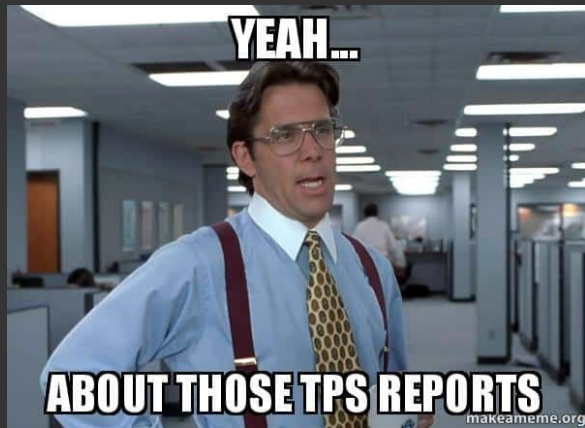


Brought to you by
[PlantUML](https://plantuml.com/)

Sequence Diagram (3)

User Requests a Report via a Web App

- The user sends a request for a report.
- The controller checks user status.
- If logged in, it processes the report request (loops through data).
- Sends confirmation (async).
- Creates a temporary object for formatting.
- Self-calls for formatting.
- Then destroys the temporary object.



Sequence Diagram (3)

@startuml

```
actor User
participant "Web Browser" as Browser
participant "Controller" as Ctrl
participant "Report Service" as Report
participant "Email Service" as Email
```

```
User -> Browser : click "Request Report"
Browser -> Ctrl : requestReport()
<<synchronous>>
activate Ctrl
```

```
Ctrl -> Ctrl : checkUserLoggedIn()
<<self-call>>
alt User is logged in
    Ctrl -> Report : generateReport()
<<synchronous>>
    activate Report
```

```
    loop For each dataset
        Report -> Report :
        processData() <<self-call>>
    end
```

```
Report --> Ctrl : reportData
deactivate Report
```

```
Ctrl -> Email : notifyUserAsync()
<<asynchronous>>
```

```
create "Formatter" as F
Ctrl -> F : new Formatter()
activate F
```

```
F -> F : format(reportData)
F --> Ctrl : formattedReport
deactivate F
destroy F
```

```
else User not logged in
    Ctrl -> Browser : showLoginPage()
end
```

```
deactivate Ctrl
Ctrl --> Browser : showResult()
```

@enduml



Brought to you by
PlantUML

State Diagram

- Order Lifecycle in an Online Store
 - This state diagram models the states an order goes through in an e-commerce platform.

States:

- **New** – Order is created.
- **Pending Payment** – Waiting for user to complete payment.
- **Paid** – Payment completed successfully.
- **Shipped** – Order has been shipped.
- **Delivered** – Order received by the customer.
- **Cancelled** – Order was cancelled by user or system.

Events that Trigger Transitions:

- `placeOrder()`
- `confirmPayment()`
- `shipOrder()`
- `deliverOrder()`
- `cancelOrder()`

State Diagram

```
@startuml
[*] --> New

New --> Pending_Payment : placeOrder()
Pending_Payment --> Paid : confirmPayment()
Pending_Payment --> Cancelled : cancelOrder()

Paid --> Shipped : shipOrder()
Shipped --> Delivered : deliverOrder()
Paid --> Cancelled : cancelOrder()
Shipped --> Cancelled : cancelOrder()

Delivered --> [*]
Cancelled --> [*]
@enduml
```



Brought to you by
[PlantUML](https://plantuml.com/)

State Diagram (2)

- Scenario: Smart Home Day/Night System
- The system controls lighting and automation behavior during Day and Night. The states handle sensor events, user overrides, and transitions between power states, with nested states, history, errors, guards, self-transitions, etc.



State Diagram (2)

```
@startuml
hide empty description

[*] --> Init

state Init {
    [*] --> Idle
    Idle --> ActiveMode : powerOn /
loadConfig()
}

state ActiveMode {

    state Day {
        Day: entry = logTime("Day starts")
        Day: do - manageDayLights()
        Day: exit - saveDayLog()
    }

    state Night {
        Night: entry - logTime("Night
starts")
        Night: do - monitorMotion()
        Night: exit - saveNightLog()

        Night --> Night : motionDetected()
/ adjustLighting()
    }

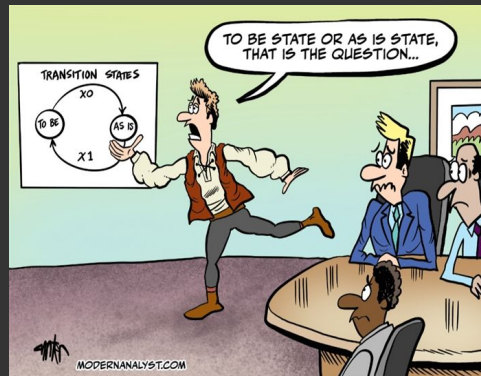
    Day --> Night : lightLevel < 20 /
switchToNight()
    Night --> Day : lightLevel > 70 /
switchToDay()
}

ActiveMode --> Standby : userAway() /
dimAll()
Standby --> ActiveMode[H*] :
userReturns()

Standby --> Error : timeout /
notifyUser()
Error --> ActiveMode : reset()

ActiveMode --> Shutdown :
userShutsDown() / closeLogs()
Shutdown --> [*] : complete

@enduml
```



Brought to you by
[PlantUML](https://plantuml.com/)

Use Case Diagram

- Let's imagine a Smart Home System where:
 - Users can interact with their homes through a mobile app.
 - Guests have limited access.
 - Technicians perform diagnostics and maintenance.



Use Case Diagram

```
@startuml
actor Homeowner
actor Guest
actor Technician

rectangle "Smart Home Control System" {
(Login)
(Control Devices)
(Schedule Automation)
(Monitor Cameras)
(Receive Alerts)
(Add/Remove Users)
(Diagnose System)
(Update Firmware)

Homeowner --> (Login)
Homeowner --> (Control Devices)
Homeowner --> (Schedule Automation)
Homeowner --> (Monitor Cameras)
Homeowner --> (Receive Alerts)
Homeowner --> (Add/Remove Users)

Guest --> (Login)
Guest --> (Control Devices)

Technician --> (Diagnose System)

(Diagnose System) <|-- (Update Firmware) : <<extend>>
(Control Devices) --> (Receive Alerts) : <<include>>
}
@enduml
```



Brought to you by
[PlantUML](https://plantuml.com/)